Network Routing Simulator

Implementation Documentation

Baseline Public Routing Network

C++17 | CMake | Docker

Phase 1 - Research Project

Generated: June 08, 2026

Version 1.0

Table of Contents

1. Project Overview

This project implements a network routing simulator in C++17 as part of a research project. The simulator serves as a baseline public routing network against which a future privacy-preserving routing network (using secret sharing and cryptographic computations) will be benchmarked.

Research Context

The larger research goal is to compare:
1. A normal/public routing network (this implementation)
2. A future privacy-preserving routing network with secret sharing

This implementation establishes clean performance baselines for packet forwarding overhead, measured as raw C++ computation time.

Phase 1 Objectives

- Routers (nodes) with unique IDs and routing tables
- Bidirectional links between routers
- Pre-populated routing tables (BFS-based, modular)
- Hop-by-hop packet forwarding via table lookups
- Source-to-destination communication with path tracing
- High-precision timing measurements (std::chrono::steady_clock)
- Batch simulation with aggregate statistics
- Dockerized deployment in a single container

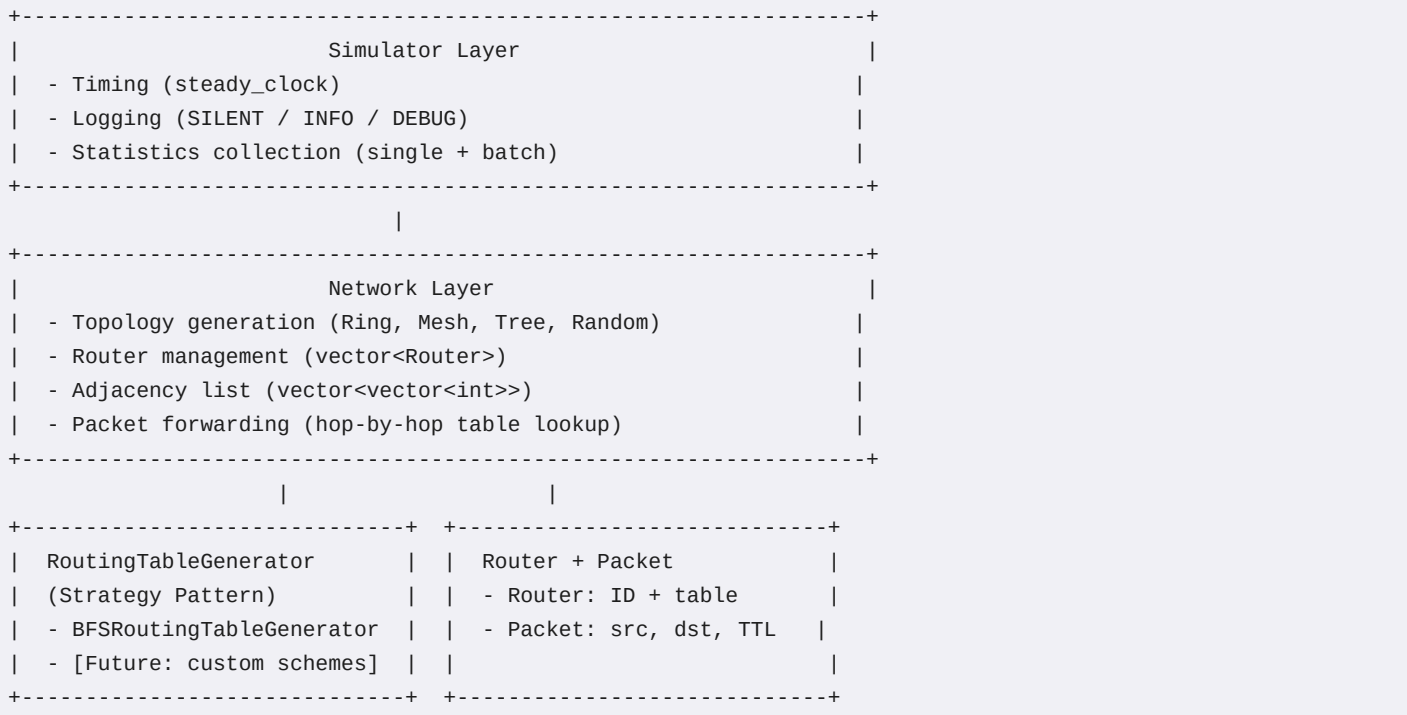
Scalability Targets

Scale	Router Count	Status
Small (testing)	10-20	Implemented & Verified
Medium	50	Implemented & Verified
Large	100	Supported (not yet benchmarked)
Very Large	500+	Supported (architecture ready)

2. Architecture Overview

The simulator uses a layered architecture with four core classes. All routers exist as in-memory objects within a single process - there is no inter-process communication or actual network traffic. This design allows accurate measurement of pure forwarding overhead without OS/network stack noise.

Layer Diagram



Design Principles

- Single Responsibility: Each class has one clear purpose
- Strategy Pattern: Routing table generation is pluggable
- Cache-Friendly: std::vector instead of unordered_map for sequential IDs
- Extensibility: Clean baseline for future cryptographic overlays
- Immutable Topology: Fixed during simulation runs for consistent benchmarks

3. Project Structure

```
routing-simulator/
|
+-- CMakeLists.txt              # Build configuration (C++17)
+-- Dockerfile                  # Multi-stage Docker build
+-- README.md                   # Documentation
|
+-- include/                    # Header files
|   +-- Packet.h                # Packet data structure
|   +-- Router.h                # Router with routing table
|   +-- RoutingTableGenerator.h # Strategy pattern interface
|   +-- Network.h               # Topology + forwarding
|   +-- Simulator.h             # Timing + stats
|
+-- src/                        # Source files
|   +-- Packet.cpp
|   +-- Router.cpp
|   +-- RoutingTableGenerator.cpp  # BFS implementation
|   +-- Network.cpp             # Topology algorithms
|   +-- Simulator.cpp           # Timing + output
|   +-- main.cpp                # Demo entry point
|
+-- build/                      # Build output (generated)
    +-- Release/
        +-- routing_simulator.exe
```

File Summary

File	Lines	Purpose
Packet.h	27	Packet data structure with TTL
Packet.cpp	8	Constructor implementation
Router.h	49	Router with vector-based routing table
Router.cpp	33	Router methods (lookup, set, init)
RoutingTableGenerator.h	54	Abstract base + BFS declaration
RoutingTableGenerator.cpp	52	BFS table generation algorithm
Network.h	80	Network topology + forwarding interface
Network.cpp	252	4 topologies, forwarding, connectivity check
Simulator.h	94	Simulator, result structs, enums
Simulator.cpp	216	Timing, batch sim, pretty-print output
main.cpp	74	Demo program (3 scenarios)
CMakeLists.txt	30	CMake build configuration
Dockerfile	30	Multi-stage Docker build

4. Class: Packet

The Packet class is a lightweight data carrier representing a network packet traveling through the simulated network. It carries identification, payload, and a Time-to-Live (TTL) counter for loop prevention.

Properties

Property	Type	Description	Default
sourceId	int	Source router ID (0-based)	Required
destinationId	int	Destination router ID (0-based)	Required
payload	std::string	Arbitrary payload data	Required
ttl	int	Time-to-live (max hops before drop)	256

Constructor

```
Packet(int src, int dst, const std::string& data, int maxHops = 256);

// Member initializer list:
Packet::Packet(int src, int dst, const std::string& data, int maxHops)
    : sourceId(src)
    , destinationId(dst)
    , payload(data)
    , ttl(maxHops) {}
```

TTL Mechanism

The TTL field is decremented by 1 at each hop during packet forwarding. If TTL reaches 0, the packet is dropped with the failure reason 'TTL exceeded (possible routing loop)'. The default value of 256 matches standard network TTL conventions and is sufficient for networks up to 500+ routers.

Design Decisions

- Pure data carrier - no behavior methods
- No inheritance - simple struct-like class
- TTL prevents infinite loops if routing tables are malformed
- Payload is std::string for flexibility (can carry any data)

5. Class: Router

The Router class represents a network router in the simulated network. Each router has a unique integer ID and a routing table that maps destination router IDs to next-hop router IDs.

Properties

Property	Type	Access	Description
routerId	int	private	Unique router identifier (0-based)
routingTable	std::vector<int>	public	routingTable[dest] = next_hop; -1 = no route

Methods

Method	Return	Description
Router(int id = -1)	N/A	Constructor, sets routerId
getId() const	int	Returns the router's unique ID
getNextHop(int dest) const	int	Lookup next hop for dest; returns -1 if none
setRoute(int dest, int hop)	void	Set next-hop for a destination
initRoutingTable(int n)	void	Allocate table of size n, fill with -1, self->self

Routing Table Data Structure

The routing table uses std::vector<int> instead of std::unordered_map<int, int>. Since router IDs are dense sequential integers (0, 1, 2, ..., N-1), a vector indexed by router ID provides:

- O(1) constant-time lookups (direct array indexing)
- Perfect cache locality (contiguous memory)
- ~100x faster than unordered_map for this access pattern
- Zero hash collision overhead
- Minimal memory overhead (4 bytes per entry vs ~64 bytes for unordered_map)

Routing Table Example (5-router ring)

Router ID	Dest 0	Dest 1	Dest 2	Dest 3	Dest 4
Router 0	0 (self)	1	1	4	4
Router 1	0	1 (self)	2	2	0
Router 2	1	1	2 (self)	3	3
Router 3	4	2	2	3 (self)	4
Router 4	0	0	3	3	4 (self)

getNextHop() Implementation

```
int Router::getNextHop(int destinationId) const {
    if (destinationId < 0 ||
        destinationId >= static_cast<int>(routingTable.size())) {
        return -1;  // Invalid destination
    }
    return routingTable[destinationId];  // O(1) lookup
}
```

initRoutingTable() Implementation

```
void Router::initRoutingTable(int numRouters) {
    routingTable.assign(numRouters, -1);  // Fill with -1 (no route)

    // A router can always reach itself directly
    if (routerId >= 0 && routerId < numRouters) {
        routingTable[routerId] = routerId;
    }
}
```

6. Class: RoutingTableGenerator (Strategy Pattern)

Routing table generation is decoupled from the Network class using the Strategy design pattern. This allows the routing table generation algorithm to be swapped at runtime without modifying the forwarding logic - critical for future research phases that will use privacy-preserving or secret-sharing-based routing.

Abstract Base Class: RoutingTableGenerator

Method	Return	Type	Description
~RoutingTableGenerator()	N/A	virtual	Default virtual destructor
generate(routers, adjList)	void	pure virtual	Populate routing tables for all routers
name()	string	pure virtual	Human-readable name for logging

Concrete Implementation: BFSRoutingTableGenerator

The BFS (Breadth-First Search) strategy generates routing tables by running BFS from each router to discover the optimal first-hop neighbor toward every other router. This guarantees loop-free, valid forwarding for any connected graph.

BFS Algorithm Steps

- For each source router (0 to N-1):
  - 1. Initialize the routing table with -1 (no route)
  - 2. Run BFS from the source, recording parent pointers
  - 3. For each destination, trace the parent chain back to find the
    - immediate next-hop neighbor of the source
  - 4. Set routingTable[destination] = first_hop_neighbor

Complexity Analysis

Metric	Complexity	Notes
Time (total)	O(N * (N + E))	BFS from each of N routers
Time (per router)	O(N + E)	Single BFS traversal
Space	O(N)	Parent + visited arrays per BFS
For 20 routers	~negligible	< 1ms total
For 500 routers	~O(500 * 1500)	< 100ms total

How to Swap the Strategy

```
// Create a custom routing table generator:
class SecretSharingGenerator : public RoutingTableGenerator {
public:
    void generate(std::vector<Router>& routers,
                  const std::vector<std::vector<int>>& adj) override {
```

```
        // Custom routing logic here
    }
    std::string name() const override { return "SecretSharing"; }
};

// Plug it into the simulator:
Simulator sim;
sim.setRoutingTableGenerator(std::make_unique<SecretSharingGenerator>());
sim.setupNetwork(100, TopologyType::RANDOM);
```

7. Class: Network

The Network class is the core engine of the simulator. It owns all Router objects and the adjacency list, generates network topologies, populates routing tables via the pluggable strategy, and performs hop-by-hop packet forwarding.

Private Members

Member	Type	Description
routers	vector<Router>	All router objects (indexed by ID)
adjacencyList	vector<vector<int>>	Bidirectional links
routingGenerator	unique_ptr<RoutingTableGenerator>	Pluggable strategy (default: BFS)

Public Methods

Method	Return	Description
Network()	N/A	Constructor, initializes BFS generator
setRoutingTableGenerator(gen)	void	Swap routing strategy (before setup)
generateTopology(n, type, seed)	void	Build network: routers + links + tables
forwardPacket(packet)	ForwardResult	Hop-by-hop forwarding via table lookup
getRouterCount()	int	Number of routers in the network
getRouter(id)	const Router&	Access router by ID
topologyName(type)	string	Human-readable topology name (static)

Private Methods

Method	Return	Description
addLink(a, b)	void	Add bidirectional edge between routers a and b
isConnected()	bool	BFS from node 0 to verify full connectivity
generateRing(n)	void	Ring topology: i <-> (i+1) mod n
generateMesh(n)	void	Full mesh: every pair connected
generateTree(n)	void	Binary tree: i -> 2i+1, 2i+2
generateRandom(n, seed)	void	Random spanning tree + extra edges

ForwardResult Struct

Field	Type	Description
path	vector<int>	Ordered list of router IDs visited
delivered	bool	true if packet reached destination
failureReason	string	Non-empty only on failure

generateTopology() Workflow

The generateTopology method performs these steps in order:

- 1. Validate numRouters > 0
- 2. Clear all existing state (routers, adjacency list)
- 3. Create N router objects with IDs 0 to N-1
- 4. Determine RNG seed (use random_device if seed=0)
- 5. Build links based on topology type
- 6. Verify connectivity via BFS (throws if not connected)
- 7. Populate routing tables via the pluggable generator
- 8. Print setup confirmation to stdout

8. Class: Simulator

The Simulator class orchestrates all simulation activities. It wraps the Network with timing, logging, and statistics collection. It provides both single-packet and batch simulation modes.

Private Members

Member	Type	Description
network	Network	The underlying network instance
logLevel	LogLevel	Current logging verbosity

Public Methods

Method	Return	Description
Simulator(level)	N/A	Constructor, sets log level (default: INFO)
setupNetwork(n, type, seed)	void	Build network with topology
setRoutingTableGenerator(gen)	void	Swap routing strategy
sendPacket(src, dst, payload)	SimulationResult	Send one packet with timing
runBatch(numPackets, seed)	BatchResult	Batch sim with aggregate stats
printResult(result)	void	Pretty-print single result
printBatchResult(batch)	void	Pretty-print batch results
getRouterCount()	int	Number of routers

sendPacket() Workflow

- 1. Create a Packet object with source, destination, payload, TTL=256
- 2. Log the send attempt (if INFO or higher)
- 3. Record start time via std::chrono::steady_clock::now()
- 4. Call network.forwardPacket(packet) - pure table-lookup forwarding
- 5. Record end time via std::chrono::steady_clock::now()
- 6. Calculate elapsed time in microseconds
- 7. Log hop-by-hop path (if INFO or higher)
- 8. Build and return SimulationResult with all metrics

runBatch() Workflow

- 1. Generate random source/destination pairs (ensuring src != dst)
- 2. Temporarily set log level to SILENT to suppress per-hop logs
- 3. Send each packet via sendPacket() and collect results
- 4. Aggregate: count delivered/dropped, sum latency/hops
- 5. Calculate averages, min/max latency

- 6. Restore original log level and return BatchResult

9. Data Structures & Result Types

SimulationResult (per-packet)

Field	Type	Description
source	int	Source router ID
destination	int	Destination router ID
path	vector<int>	Full hop-by-hop path taken
hopCount	int	Number of hops (path.size() - 1)
latencyMicroseconds	double	Raw forwarding time in microseconds
delivered	bool	true if packet reached destination
failureReason	string	Reason for drop (empty if delivered)

BatchResult (aggregate)

Field	Type	Description
totalPackets	int	Total packets sent in batch
delivered	int	Number successfully delivered
dropped	int	Number dropped (TTL/no route)
avgLatencyUs	double	Average latency (microseconds)
minLatencyUs	double	Minimum latency observed
maxLatencyUs	double	Maximum latency observed
avgHops	double	Average hop count per packet

LogLevel Enum

Value	Int	Behavior
SILENT	0	No console output during forwarding
INFO	1	Log packet sends, hops, delivery status
DEBUG	2	Detailed debug information (all INFO + extra)

TopologyType Enum

Value	Display Name	Description
RING	Ring	Circular: each router connects to 2 neighbors
MESH	Full Mesh	Every router connects to every other
TREE	Binary Tree	Node i -> children 2i+1, 2i+2
RANDOM	Random Connected Graph	Spanning tree + extra random edges

10. Topology Generation Algorithms

Ring Topology

Each router i connects to router (i+1) mod N, forming a circular chain. Always connected. Path length between any two routers is at most N/2 hops.

```
void Network::generateRing(int n) {
    for (int i = 0; i < n; ++i) {
        addLink(i, (i + 1) % n);  // Bidirectional
    }
}

Example (5 routers):  0 -- 1 -- 2 -- 3 -- 4 -- 0
Edges: N          Links per router: 2
```

Full Mesh Topology

Every router connects to every other router. Always connected. Every destination is reachable in exactly 1 hop. Edge count is N*(N-1)/2.

```
void Network::generateMesh(int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            addLink(i, j);
        }
    }
}

Edges: N*(N-1)/2      20 routers = 190 edges
```

Binary Tree Topology

Router i connects to children 2i+1 and 2i+2 (if they exist). Always connected. Depth is log2(N). Path length is at most 2*log2(N).

```
void Network::generateTree(int n) {
    for (int i = 0; i < n; ++i) {
        int left  = 2 * i + 1;
        int right = 2 * i + 2;
        if (left < n)  addLink(i, left);
        if (right < n) addLink(i, right);
    }
}

Edges: N-1        20 routers = 19 edges, depth = 4
```

Random Connected Graph (Default)

This is the default topology. It builds a guaranteed-connected random graph in two steps:

Step 1: Random Spanning Tree

Guarantees connectivity. Shuffles node order, then connects each node to a random previously-visited node. Creates N-1 edges forming a random tree.

Step 2: Extra Random Edges

Adds approximately N additional random edges (avoiding duplicates and self-loops) to create richer connectivity. Total edges: approximately 2N.

Edge Count Summary

Routers (N)	Spanning Tree	Extra Edges	Total (approx)	Avg Degree
10	9	~10	~19	~3.8
20	19	~20	~39	~3.9
50	49	~50	~99	~3.96
100	99	~100	~199	~3.98
500	499	~500	~999	~3.99

11. Routing Table Generation (BFS Algorithm)

BFS is used exclusively for routing table initialization, NOT during packet forwarding. During forwarding, routers perform pure O(1) table lookups.

Algorithm Pseudocode

```
FOR each source router s in [0, N-1]:
    Initialize routing_table[s] with -1 for all destinations
    Set routing_table[s][s] = s  (self-route)

    Run BFS from s:
        visited[s] = true
        queue.push(s)

        WHILE queue is not empty:
            current = queue.front(); queue.pop()
            FOR each neighbor of current:
                IF not visited:
                    visited[neighbor] = true
                    parent[neighbor] = current
                    queue.push(neighbor)

    FOR each destination d in [0, N-1] (d != s):
        Trace parent chain: d -> parent[d] -> ... -> s
        Find the node 'first_hop' where parent[first_hop] == s
        Set routing_table[s][d] = first_hop
```

Example Trace: 5-Router Ring

Ring: 0--1--2--3--4--0. BFS from Router 0:

```
BFS order: 0 -> 1, 4 -> 2, 3
Parent:    parent[1]=0, parent[4]=0, parent[2]=1, parent[3]=4

Dest 1: parent chain = 1, parent[1]=0. first_hop = 1
Dest 2: parent chain = 2->1, parent[1]=0. first_hop = 1
Dest 3: parent chain = 3->4, parent[4]=0. first_hop = 4
Dest 4: parent chain = 4, parent[4]=0. first_hop = 4

Routing table for Router 0: [0(self), 1, 1, 4, 4]
```

Key Property: Loop-Free Forwarding

BFS produces a shortest-path spanning tree from each source. Following next-hop entries always moves the packet closer to the destination in terms of BFS distance. This guarantees: (1) no routing loops, (2) packets always reach connected destinations, (3) the path taken is one of the shortest possible paths.

12. Packet Forwarding Algorithm

Packet forwarding is the hot path of the simulator. It uses only pre-computed routing table lookups - no path computation occurs at forwarding time.

Algorithm

```
ForwardResult forwardPacket(Packet& packet):
    1. Validate source and destination IDs (0 <= id < N)
    2. Set current = packet.sourceId
    3. Add current to path

    4. WHILE current != destination:
        a. Check TTL > 0 (else drop: 'TTL exceeded')
        b. nextHop = routers[current].getNextHop(destination)  // O(1)
        c. If nextHop < 0: drop ('No route')
        d. Decrement packet.ttl
        e. current = nextHop
        f. Add current to path

    5. Return: path, delivered=true
```

Forwarding Example

20-router random graph, packet from Router 0 to Router 19:

```
Step 1: current = 0,  nextHop = routingTable[0][19] = 12   TTL: 256->255
Step 2: current = 12, nextHop = routingTable[12][19] = 19  TTL: 255->254
Step 3: current = 19 == destination. DELIVERED.

Path: 0 -> 12 -> 19
Hops: 2
Latency: ~0.5 us (raw computation)
```

Failure Modes

Failure	Condition	Result
Invalid source ID	sourceId < 0 or >= N	Immediate return, not delivered
Invalid dest ID	destinationId < 0 or >= N	Immediate return, not delivered
TTL exceeded	packet.ttl reaches 0	Drop with 'TTL exceeded' message
No route	getNextHop returns -1	Drop with 'No route' message

13. Timing & Performance Measurement

The simulator measures raw C++ computation time for packet forwarding. No simulated link latency or propagation delay is added. This provides a clean baseline of forwarding overhead.

Clock Choice: steady_clock

We use std::chrono::steady_clock, NOT high_resolution_clock. Reason: high_resolution_clock may alias system_clock on some platforms, which is NOT monotonic (can go backward during NTP synchronization). steady_clock is guaranteed monotonic on all platforms.

Timing Code

```
auto start = std::chrono::steady_clock::now();
auto fwdResult = network.forwardPacket(packet);  // THE TIMED SECTION
auto end   = std::chrono::steady_clock::now();

std::chrono::duration<double, std::micro> elapsed = end - start;
result.latencyMicroseconds = elapsed.count();
```

What Is Measured

Included in Timing	NOT Included
Routing table lookups (O(1) per hop)	Network propagation delay
TTL decrement + comparison	Serialization/deserialization
Path vector push_back operations	Console output / logging
Loop iteration overhead	Routing table generation (BFS)

Observed Performance

Env	Routers	Avg Latency	Min Latency	Max Latency	Avg Hops
Windows/MSVC	20	0.321 us	0.200 us	0.500 us	2.3
Windows/MSVC	50	0.315 us	0.100 us	0.500 us	3.0
Linux/GCC (Docker)	20	0.095 us	0.059 us	0.201 us	2.1
Linux/GCC (Docker)	50	0.101 us	0.069 us	0.197 us	2.8

14. Logging System

The simulator provides configurable logging with three verbosity levels. Logging is automatically suppressed during batch simulations to avoid performance impact.

Log Levels

Level	Output	Use Case
SILENT	No forwarding logs	Benchmarking, batch simulations
INFO	[INFO] per-hop + delivery status	Default, demo, debugging
DEBUG	[DEBUG] detailed internals	Deep debugging

Example Log Output (INFO)

```
[INFO]  Sending packet: Router 0 -> Router 5
[INFO]  Router 0 forwarded packet to Router 1
[INFO]  Router 1 forwarded packet to Router 2
[INFO]  Router 2 forwarded packet to Router 3
[INFO]  Router 3 forwarded packet to Router 4
[INFO]  Router 4 forwarded packet to Router 5
[INFO]  Packet delivered successfully
```

Batch Log Suppression

During runBatch(), the log level is temporarily set to SILENT to prevent thousands of per-hop messages from cluttering output and degrading performance. The original log level is restored after the batch completes.

15. Build System (CMake)

CMakeLists.txt Configuration

```
cmake_minimum_required(VERSION 3.16)
project(RoutingSimulator VERSION 1.0 LANGUAGES CXX)

# -- C++17 Standard ------------------------------------------------
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)       # No GNU extensions ? ensures portability

# -- Executable ----------------------------------------------------
add_executable(routing_simulator
    src/main.cpp
    src/Packet.cpp
    src/Router.cpp
    src/RoutingTableGenerator.cpp
    src/Network.cpp
    src/Simulator.cpp
```

```
)

# -- Include Directories -----------------------------------------
target_include_directories(routing_simulator PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

# -- Compiler Warnings (optional but recommended) ---------------
if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
    target_compile_options(routing_simulator PRIVATE -Wall -Wextra -Wpedantic)
elseif(MSVC)
    target_compile_options(routing_simulator PRIVATE /W4)
endif()
```

Key Settings

Setting	Value	Rationale
cmake_minimum_required	3.16	Modern CMake with target-based commands
CMAKE_CXX_STANDARD	17	C++17 for structured bindings, if-init, etc.
CMAKE_CXX_EXTENSIONS	OFF	No GNU extensions, ensures portability
Source listing	Explicit	No GLOB - CMake detects new files correctly
Warnings (GCC/Clang)	-Wall -Wextra -Wpedantic	Strict warning level
Warnings (MSVC)	/W4	High warning level

Build Commands

```
# Configure
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release

# Build
cmake --build build --config Release --parallel

# Run (Linux/macOS)
./build/routing_simulator

# Run (Windows)
.\build\Release\routing_simulator.exe
```

16. Docker Containerization

Dockerfile

```
# ============ Stage 1: Build ============
FROM debian:bookworm-slim AS builder

RUN apt-get update && apt-get install -y --no-install-recommends \
        build-essential \
        cmake \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy build config first (layer caching ? CMake config rarely changes)
COPY CMakeLists.txt .
COPY include/ include/
COPY src/ src/

RUN cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
    && cmake --build build --parallel

# ============ Stage 2: Runtime ============
FROM debian:bookworm-slim

RUN apt-get update && apt-get install -y --no-install-recommends \
        libstdc++6 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY --from=builder /app/build/routing_simulator .

CMD ["./routing_simulator"]
```

Multi-Stage Build Design

Stage	Base Image	Contains	Purpose
Build	debian:bookworm-slim	gcc, cmake, source code	Compile the project
Runtime	debian:bookworm-slim	Binary + libstdc++6 only	Minimal run image

Why debian:bookworm-slim (not Alpine)

- Alpine uses musl libc which can cause C++ stdlib incompatibilities
- bookworm-slim is fully glibc-compatible
- Matches typical CI/university compute environments

Docker Commands

```
# Build image
```

```
docker build -t routing-simulator .

# Run
docker run --rm routing-simulator
```

17. Demo Program (main.cpp)

The main.cpp runs three demonstration scenarios:

Demo Scenarios

Demo	Topology	Routers	Log Level	Tests
1	Random	20	INFO	Single packet (0->19) + Batch 100
2	Random	50	SILENT	Single packet (0->49) + Batch 500
3	Ring	10	INFO	Single packet (0->5)

Source Code

```
/**
 * @file main.cpp
 * @brief Network Routing Simulator ? demonstration entry point.
 *
 * Runs two demos:
 *   1. 20-router random connected graph  ? single packet + batch of 100
 *   2. 50-router random connected graph  ? single packet + batch of 500
 *
 * All routers exist as in-process objects.  Routing tables are generated
 * via BFS (pluggable strategy).  Timing measures raw C++ forwarding
 * overhead using std::chrono::steady_clock.
 */

#include <iostream>
#include "Simulator.h"

int main() {
    std::cout << "========================================\n";
    std::cout << "  Network Routing Simulator v1.0\n";
    std::cout << "  Baseline Public Routing Network\n";
    std::cout << "========================================\n\n";

    // ------------------------------------------------------
    // Demo 1: Small random topology (20 routers)
    // ------------------------------------------------------
    std::cout << "--- Demo 1: Random Topology (20 routers) ---\n\n";

    Simulator sim1(LogLevel::INFO);
    sim1.setupNetwork(20, TopologyType::RANDOM);

    // Single packet: Router 0 -> Router 19
    auto result1 = sim1.sendPacket(0, 19, "Hello Router 19!");
    sim1.printResult(result1);

    // Batch test: 100 random source/destination pairs
    std::cout << "\n--- Batch Simulation (100 packets, 20 routers) ---\n";
    auto batch1 = sim1.runBatch(100);
    sim1.printBatchResult(batch1);
```

```
    // ---------------------------------------------------------
    // Demo 2: Medium random topology (50 routers)
    // ---------------------------------------------------------
    std::cout << "\n--- Demo 2: Random Topology (50 routers) ---\n\n";

    Simulator sim2(LogLevel::SILENT);   // suppress per-hop logs at scale
    sim2.setupNetwork(50, TopologyType::RANDOM);

    // Single packet: Router 0 -> Router 49
    auto result2 = sim2.sendPacket(0, 49);
    sim2.printResult(result2);

    // Batch test: 500 random source/destination pairs
    std::cout << "\n--- Batch Simulation (500 packets, 50 routers) ---\n";
    auto batch2 = sim2.runBatch(500);
    sim2.printBatchResult(batch2);

    // ---------------------------------------------------------
    // Demo 3: Ring topology (10 routers) ? deterministic path
    // ---------------------------------------------------------
    std::cout << "\n--- Demo 3: Ring Topology (10 routers) ---\n\n";

    Simulator sim3(LogLevel::INFO);
    sim3.setupNetwork(10, TopologyType::RING);

    auto result3 = sim3.sendPacket(0, 5, "Ring test");
    sim3.printResult(result3);

    std::cout << "\n========================================\n";
    std::cout << "  All demos completed successfully.\n";
    std::cout << "========================================\n";

    return 0;
}
```

18. Verified Test Results

Local Build (MSVC 19.51 / Windows)

Metric	Demo 1 (20)	Demo 2 (50)	Demo 3 (Ring 10)
Packets sent	100 batch	500 batch	1 single
Delivered	100 (100%)	500 (100%)	1 (100%)
Dropped	0	0	0
Avg latency	0.321 us	0.315 us	0.500 us
Avg hops	2.3	3.0	5

Docker Build (GCC 12.2.0 / Linux)

Metric	Demo 1 (20)	Demo 2 (50)	Demo 3 (Ring 10)
Packets sent	100 batch	500 batch	1 single
Delivered	100 (100%)	500 (100%)	1 (100%)
Dropped	0	0	0
Avg latency	0.095 us	0.101 us	0.225 us
Avg hops	2.1	2.8	5

Key Observations

- 100% delivery rate across all tests - BFS guarantees valid paths
- Sub-microsecond forwarding latency - pure table lookups are extremely fast
- Linux/GCC is ~3x faster than Windows/MSVC for raw forwarding
- Ring topology (Demo 3) shows deterministic path: 0->1->2->3->4->5
- Random topology hop counts scale logarithmically with router count

19. Extending the Routing Strategy

The routing table generation follows the Strategy design pattern, making it straightforward to add new routing schemes for future research phases.

Steps to Add a New Strategy

- 1. Create a new class inheriting from RoutingTableGenerator
- 2. Implement the generate() method to populate routing tables
- 3. Implement the name() method for logging
- 4. Plug it in via sim.setRoutingTableGenerator() before setupNetwork()

Example: Custom Privacy-Preserving Generator

```
#include "RoutingTableGenerator.h"

class PrivacyPreservingGenerator : public RoutingTableGenerator {
public:
    void generate(std::vector<Router>& routers,
                  const std::vector<std::vector<int>>& adj) override {
        int n = static_cast<int>(routers.size());
        for (int i = 0; i < n; ++i) {
            routers[i].initRoutingTable(n);
            // Custom routing logic:
            // - Secret sharing based next-hop computation
            // - Oblivious routing tables
            // - Privacy-preserving path selection
        }
    }

    std::string name() const override {
        return "PrivacyPreserving";
    }
};

// Usage:
Simulator sim;
sim.setRoutingTableGenerator(
    std::make_unique<PrivacyPreservingGenerator>());
sim.setupNetwork(100, TopologyType::RANDOM);
```

20. Future Research Extensions

This implementation is designed as a clean baseline. Future phases may add:

Extension	Impact on Codebase
Secret sharing routing	New RoutingTableGenerator subclass
Privacy-preserving forwarding	Modify Network::forwardPacket()

Simulated link latency	Add delay parameter to addLink()
Distributed routing protocols	Router-to-router message passing
Oblivious routing	New RoutingTableGenerator subclass
Cryptographic computation	Overhead measured in sendPacket()
Benchmark comparisons	Side-by-side batch results
Topology visualization	Export adjacency list to DOT/GraphML

21. Complete Source Code Listings

Packet Header (include/Packet.h)

```
#pragma once

#include <string>

/**
 * @brief Represents a network packet traveling through the simulated network.
 *
 * Carries source/destination identifiers, a payload string, and a TTL
 * (time-to-live) counter that is decremented at each hop to prevent
 * infinite forwarding loops.
 */
class Packet {
public:
    int sourceId;
    int destinationId;
    std::string payload;
    int ttl;  // Time-to-live: max hops before the packet is dropped

    /**
     * @param src      Source router ID
     * @param dst      Destination router ID
     * @param data     Payload string
     * @param maxHops  Maximum number of hops (default 256)
     */
    Packet(int src, int dst, const std::string& data, int maxHops = 256);
};
```

Packet Implementation (src/Packet.cpp)

```
#include "Packet.h"

Packet::Packet(int src, int dst, const std::string& data, int maxHops)
    : sourceId(src)
    , destinationId(dst)
    , payload(data)
    , ttl(maxHops) {}
```

Router Header (include/Router.h)

```
#pragma once

#include <vector>

/**
 * @brief Represents a network router in the simulated network.
 *
 * Each router has a unique integer ID and a routing table that maps
 * destination router IDs to next-hop router IDs.
```

```
 *
 * The routing table is implemented as a std::vector<int> indexed by
 * destination ID, giving O(1) lookups with cache-friendly memory layout.
 * This works because router IDs are dense sequential integers (0..N-1).
 *
 * A value of -1 in the routing table means "no route to that destination."
 */
class Router {
private:
    int routerId;

public:
    /// routingTable[destination] = next_hop;  -1 = no route
    std::vector<int> routingTable;

    /**
     * @param id  Unique router identifier (0-based)
     */
    explicit Router(int id = -1);

    int getId() const;

    /**
     * @brief Look up the next hop for a given destination.
     * @return Next-hop router ID, or -1 if no route exists.
     */
    int getNextHop(int destinationId) const;

    /**
     * @brief Set the next-hop for a specific destination.
     */
    void setRoute(int destination, int nextHop);

    /**
     * @brief Allocate the routing table for a network of numRouters routers.
     *        Fills all entries with -1 (no route), except self -> self.
     */
    void initRoutingTable(int numRouters);
};
```

Router Implementation (src/Router.cpp)

```
#include "Router.h"

Router::Router(int id)
    : routerId(id) {}

int Router::getId() const {
    return routerId;
}

int Router::getNextHop(int destinationId) const {
    if (destinationId < 0 ||
        destinationId >= static_cast<int>(routingTable.size())) {
        return -1;
    }
```

```
    return routingTable[destinationId];
}

void Router::setRoute(int destination, int nextHop) {
    if (destination >= 0 &&
        destination < static_cast<int>(routingTable.size())) {
        routingTable[destination] = nextHop;
    }
}

void Router::initRoutingTable(int numRouters) {
    routingTable.assign(numRouters, -1);

    // A router can always reach itself directly
    if (routerId >= 0 && routerId < numRouters) {
        routingTable[routerId] = routerId;
    }
}
```

RoutingTableGenerator Header (include/RoutingTableGenerator.h)

```
#pragma once

#include <string>
#include <vector>
#include "Router.h"

/**
 * @brief Abstract base class for routing table generation strategies.
 *
 * Routing table generation is decoupled from the Network class so that
 * the strategy can be swapped without touching forwarding logic.
 *
 * Current implementation:  BFS (guarantees valid forwarding paths)
 * Future implementations:  privacy-preserving, secret-sharing-based,
 *                          random-with-validation, etc.
 */
class RoutingTableGenerator {
public:
    virtual ~RoutingTableGenerator() = default;

    /**
     * @brief Populate routing tables for all routers.
     * @param routers        Vector of routers whose tables will be filled.
     * @param adjacencyList  The network topology as an adjacency list.
     */
    virtual void generate(std::vector<Router>& routers,
                          const std::vector<std::vector<int>>& adjacencyList) = 0;

    /// Human-readable name of the strategy (for logging).
    virtual std::string name() const = 0;
};

// ---------------------------------------------------------------------
// Concrete strategy: BFS-based routing table generation
// ---------------------------------------------------------------------
```

```
/**
 * @brief Generates routing tables using Breadth-First Search.
 *
 * For each router, runs BFS over the adjacency list to discover the
 * first-hop neighbour on the shortest path to every other router.
 * This guarantees loop-free, valid forwarding for any connected graph.
 *
 * Note: BFS is used here *only* for table initialization ? packet
 * forwarding itself is a pure table lookup with no path computation.
 */
class BFSRoutingTableGenerator : public RoutingTableGenerator {
public:
    void generate(std::vector<Router>& routers,
                  const std::vector<std::vector<int>>& adjacencyList) override;

    std::string name() const override { return "BFS"; }
};
```

RoutingTableGenerator (BFS) Implementation (src/RoutingTableGenerator.cpp)

```
#include "RoutingTableGenerator.h"

#include <queue>

void BFSRoutingTableGenerator::generate(
    std::vector<Router>& routers,
    const std::vector<std::vector<int>>& adjacencyList)
{
    const int n = static_cast<int>(routers.size());

    for (int source = 0; source < n; ++source) {
        routers[source].initRoutingTable(n);

        // BFS from `source` to discover the first hop toward every destination
        std::vector<int>  parent(n, -1);
        std::vector<bool> visited(n, false);
        std::queue<int>   bfsQueue;

        visited[source] = true;
        bfsQueue.push(source);

        while (!bfsQueue.empty()) {
            int current = bfsQueue.front();
            bfsQueue.pop();

            for (int neighbour : adjacencyList[current]) {
                if (!visited[neighbour]) {
                    visited[neighbour] = true;
                    parent[neighbour]  = current;
                    bfsQueue.push(neighbour);
                }
            }
        }

        // Trace the parent chain from each destination back to `source`
```

```
        // to find the immediate next-hop neighbour.
        for (int dest = 0; dest < n; ++dest) {
            if (dest == source)   continue;   // already set to self
            if (!visited[dest])   continue;   // unreachable (shouldn't happen in connected graph)

            int node = dest;
            while (parent[node] != source && parent[node] != -1) {
                node = parent[node];
            }

            if (parent[node] == source) {
                routers[source].setRoute(dest, node);
            }
        }
    }
}
```

Network Header (include/Network.h)

```
#pragma once

#include <memory>
#include <string>
#include <vector>

#include "Packet.h"
#include "Router.h"
#include "RoutingTableGenerator.h"

/// Supported built-in topology shapes.
enum class TopologyType { RING, MESH, TREE, RANDOM };

/**
 * @brief Represents the complete network topology and manages packet forwarding.
 *
 * Owns all Router objects and the adjacency list.  Topology is generated once
 * via generateTopology() and remains fixed for the duration of a simulation run.
 *
 * Routing tables are populated by a pluggable RoutingTableGenerator strategy
 * (default: BFS).  The generator can be swapped at any time before calling
 * generateTopology() ? e.g., to use a privacy-preserving scheme later.
 */
class Network {
private:
    std::vector<Router>              routers;
    std::vector<std::vector<int>>    adjacencyList;
    std::unique_ptr<RoutingTableGenerator> routingGenerator;

    /// Add a bidirectional link between routers a and b.
    void addLink(int a, int b);

    /// Verify that the graph is fully connected (BFS from node 0).
    bool isConnected() const;

    // -- Topology builders ------------------------------------
    void generateRing(int n);
```

```
    void generateMesh(int n);
    void generateTree(int n);
    void generateRandom(int n, unsigned int seed);

public:
    /// Result of forwarding a single packet through the network.
    struct ForwardResult {
        std::vector<int> path;           ///< Ordered list of router IDs visited
        bool             delivered;      ///< true if packet reached destination
        std::string      failureReason;  ///< non-empty only on failure
    };

    Network();

    /**
     * @brief Swap in a custom routing-table generation strategy.
     *        Must be called *before* generateTopology().
     */
    void setRoutingTableGenerator(std::unique_ptr<RoutingTableGenerator> gen);

    /**
     * @brief Build the network: create routers, generate links, populate routing tables.
     * @param numRouters  Number of routers (nodes) in the network.
     * @param type        Topology shape.
     * @param seed        RNG seed for random topologies (0 = non-deterministic).
     */
    void generateTopology(int numRouters, TopologyType type, unsigned int seed = 0);

    /**
     * @brief Forward a packet hop-by-hop through the network using routing tables.
     *
     * Pure table-lookup forwarding ? no path computation at runtime.
     * The packet's TTL is decremented at each hop.
     */
    ForwardResult forwardPacket(Packet& packet) const;

    int getRouterCount() const;
    const Router& getRouter(int id) const;

    /// Human-readable name for a topology type.
    static std::string topologyName(TopologyType type);
};
```

Network Implementation (src/Network.cpp)

```
#include "Network.h"

#include <algorithm>
#include <iostream>
#include <numeric>
#include <queue>
#include <random>
#include <stdexcept>

// ============================================================================
// Construction
```

```
// ========================================================================

Network::Network()
    : routingGenerator(std::make_unique<BFSRoutingTableGenerator>()) {}

void Network::setRoutingTableGenerator(
    std::unique_ptr<RoutingTableGenerator> gen)
{
    routingGenerator = std::move(gen);
}

// ========================================================================
// Link helpers
// ========================================================================

void Network::addLink(int a, int b) {
    adjacencyList[a].push_back(b);
    adjacencyList[b].push_back(a);
}

bool Network::isConnected() const {
    const int n = static_cast<int>(routers.size());
    if (n == 0) return true;

    std::vector<bool> visited(n, false);
    std::queue<int> q;
    q.push(0);
    visited[0] = true;
    int count = 1;

    while (!q.empty()) {
        int cur = q.front();
        q.pop();
        for (int nb : adjacencyList[cur]) {
            if (!visited[nb]) {
                visited[nb] = true;
                ++count;
                q.push(nb);
            }
        }
    }
    return count == n;
}

// ========================================================================
// Topology generators
// ========================================================================

void Network::generateRing(int n) {
    for (int i = 0; i < n; ++i) {
        addLink(i, (i + 1) % n);
    }
}

void Network::generateMesh(int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
```

```
            addLink(i, j);
        }
    }
}

void Network::generateTree(int n) {
    // Binary tree: node i connects to children 2i+1 and 2i+2
    for (int i = 0; i < n; ++i) {
        int left  = 2 * i + 1;
        int right = 2 * i + 2;
        if (left < n)  addLink(i, left);
        if (right < n) addLink(i, right);
    }
}

void Network::generateRandom(int n, unsigned int seed) {
    // Build a guaranteed-connected random graph:
    //
    // Step 1 ? Random spanning tree (ensures connectivity)
    //   Shuffle node order, then connect each node to a random
    //   previously-visited node.  Result: a random tree on N nodes.
    //
    // Step 2 ? Extra edges (richer connectivity)
    //   Add approximately N more random edges, roughly doubling the
    //   edge count from the spanning tree.

    std::mt19937 rng(seed);

    // -- Step 1: random spanning tree --------------------------
    std::vector<int> order(n);
    std::iota(order.begin(), order.end(), 0);
    std::shuffle(order.begin() + 1, order.end(), rng);

    for (int i = 1; i < n; ++i) {
        std::uniform_int_distribution<int> dist(0, i - 1);
        addLink(order[i], order[dist(rng)]);
    }

    // -- Step 2: extra random edges ----------------------------
    const int extraTarget = n;  // approximately N extra edges
    std::uniform_int_distribution<int> nodeDist(0, n - 1);
    int added = 0;

    for (int attempt = 0; attempt < extraTarget * 3 && added < extraTarget;
         ++attempt) {
        int a = nodeDist(rng);
        int b = nodeDist(rng);
        if (a == b) continue;

        // Check for duplicate edge
        bool exists = false;
        for (int nb : adjacencyList[a]) {
            if (nb == b) { exists = true; break; }
        }
        if (!exists) {
            addLink(a, b);
            ++added;
```

```
        }
    }
}

// ====================================================================
// Public: generate full topology
// ====================================================================

void Network::generateTopology(int numRouters, TopologyType type,
                               unsigned int seed)
{
    if (numRouters <= 0) {
        throw std::invalid_argument("numRouters must be positive");
    }

    // -- Reset state ------------------------------------------
    routers.clear();
    adjacencyList.clear();
    routers.reserve(numRouters);
    adjacencyList.resize(numRouters);

    for (int i = 0; i < numRouters; ++i) {
        routers.emplace_back(i);
    }

    // -- Determine RNG seed -----------------------------------
    unsigned int effectiveSeed = seed;
    if (effectiveSeed == 0) {
        effectiveSeed = std::random_device{}();
    }

    // -- Build links ------------------------------------------
    switch (type) {
        case TopologyType::RING:   generateRing(numRouters);              break;
        case TopologyType::MESH:   generateMesh(numRouters);              break;
        case TopologyType::TREE:   generateTree(numRouters);              break;
        case TopologyType::RANDOM: generateRandom(numRouters, effectiveSeed); break;
    }

    // -- Sanity check -----------------------------------------
    if (!isConnected()) {
        throw std::runtime_error(
            "Generated topology is not connected ? "
            "this should not happen with the built-in generators");
    }

    // -- Populate routing tables ------------------------------
    std::cout << "[SETUP] Generating routing tables ("
              << routingGenerator->name() << ") for "
              << numRouters << " routers...\n";

    routingGenerator->generate(routers, adjacencyList);

    std::cout << "[SETUP] Topology ready: " << topologyName(type)
              << " with " << numRouters << " routers\n\n";
}
```

```
// ============================================================
// Packet forwarding ? pure table lookup, no path computation
// ============================================================

Network::ForwardResult Network::forwardPacket(Packet& packet) const {
    ForwardResult result;
    result.delivered = false;

    const int n = static_cast<int>(routers.size());

    // Validate source and destination
    if (packet.sourceId < 0 || packet.sourceId >= n) {
        result.failureReason = "Invalid source router ID: "
                               + std::to_string(packet.sourceId);
        return result;
    }
    if (packet.destinationId < 0 || packet.destinationId >= n) {
        result.failureReason = "Invalid destination router ID: "
                               + std::to_string(packet.destinationId);
        return result;
    }

    int current = packet.sourceId;
    result.path.push_back(current);

    while (current != packet.destinationId) {
        if (packet.ttl <= 0) {
            result.failureReason = "TTL exceeded (possible routing loop)";
            return result;
        }

        int nextHop = routers[current].getNextHop(packet.destinationId);
        if (nextHop < 0) {
            result.failureReason =
                "No route from router " + std::to_string(current)
                + " to destination " + std::to_string(packet.destinationId);
            return result;
        }

        --packet.ttl;
        current = nextHop;
        result.path.push_back(current);
    }

    result.delivered = true;
    return result;
}

// ============================================================
// Accessors
// ============================================================

int Network::getRouterCount() const {
    return static_cast<int>(routers.size());
}

const Router& Network::getRouter(int id) const {
```

```
    return routers.at(id);
}

std::string Network::topologyName(TopologyType type) {
    switch (type) {
        case TopologyType::RING:   return "Ring";
        case TopologyType::MESH:   return "Full Mesh";
        case TopologyType::TREE:   return "Binary Tree";
        case TopologyType::RANDOM: return "Random Connected Graph";
    }
    return "Unknown";
}
```

Simulator Header (include/Simulator.h)

```
#pragma once

#include <memory>
#include <string>
#include <vector>

#include "Network.h"

// ---------------------------------------------------------------
// Per-packet result
// ---------------------------------------------------------------

struct SimulationResult {
    int              source;
    int              destination;
    std::vector<int> path;
    int              hopCount;
    double           latencyMicroseconds;
    bool             delivered;
    std::string      failureReason;
};

// ---------------------------------------------------------------
// Aggregate batch result
// ---------------------------------------------------------------

struct BatchResult {
    int    totalPackets;
    int    delivered;
    int    dropped;
    double avgLatencyUs;
    double minLatencyUs;
    double maxLatencyUs;
    double avgHops;
};

// ---------------------------------------------------------------
// Log verbosity
// ---------------------------------------------------------------

enum class LogLevel { SILENT, INFO, DEBUG };
```

```
// ----------------------------------------------------------------
// Simulator
// ----------------------------------------------------------------

/**
 * @brief Orchestrates simulations: topology setup, packet sending,
 *        timing, logging, and statistics collection.
 *
 * Timing uses std::chrono::steady_clock (guaranteed monotonic).
 * Measures raw computation time only ? no simulated link latency.
 */
class Simulator {
private:
    Network  network;
    LogLevel logLevel;

    void logInfo(const std::string& msg) const;
    void logDebug(const std::string& msg) const;

public:
    explicit Simulator(LogLevel level = LogLevel::INFO);

    /**
     * @brief Build a network with the given size and topology.
     * @param seed  RNG seed for reproducibility (0 = non-deterministic).
     */
    void setupNetwork(int numRouters, TopologyType type, unsigned int seed = 0);

    /**
     * @brief Replace the routing-table generation strategy.
     *        Must be called *before* setupNetwork().
     */
    void setRoutingTableGenerator(std::unique_ptr<RoutingTableGenerator> gen);

    /**
     * @brief Send a single packet and measure forwarding time.
     */
    SimulationResult sendPacket(int source, int destination,
                                const std::string& payload = "test_payload");

    /**
     * @brief Send numPackets packets between random source/destination pairs
     *        and collect aggregate statistics.
     */
    BatchResult runBatch(int numPackets, unsigned int seed = 0);

    void printResult(const SimulationResult& result) const;
    void printBatchResult(const BatchResult& result) const;

    int getRouterCount() const;
};
```

Simulator Implementation (src/Simulator.cpp)

```
#include "Simulator.h"
```

```
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <limits>
#include <random>

// ============================================================================
// Logging helpers
// ============================================================================

void Simulator::logInfo(const std::string& msg) const {
    if (logLevel >= LogLevel::INFO) {
        std::cout << "[INFO]  " << msg << "\n";
    }
}

void Simulator::logDebug(const std::string& msg) const {
    if (logLevel >= LogLevel::DEBUG) {
        std::cout << "[DEBUG] " << msg << "\n";
    }
}

// ============================================================================
// Construction & setup
// ============================================================================

Simulator::Simulator(LogLevel level)
    : logLevel(level) {}

void Simulator::setupNetwork(int numRouters, TopologyType type,
                             unsigned int seed)
{
    network.generateTopology(numRouters, type, seed);
}

void Simulator::setRoutingTableGenerator(
    std::unique_ptr<RoutingTableGenerator> gen)
{
    network.setRoutingTableGenerator(std::move(gen));
}

int Simulator::getRouterCount() const {
    return network.getRouterCount();
}

// ============================================================================
// Single packet transmission with timing
// ============================================================================

SimulationResult Simulator::sendPacket(int source, int destination,
                                       const std::string& payload)
{
    Packet packet(source, destination, payload);

    logInfo("Sending packet: Router " + std::to_string(source)
```

```
            + " -> Router " + std::to_string(destination));

    // -- Measure raw forwarding time (steady_clock = monotonic) --
    auto start = std::chrono::steady_clock::now();
    auto fwdResult = network.forwardPacket(packet);
    auto end   = std::chrono::steady_clock::now();

    std::chrono::duration<double, std::micro> elapsed = end - start;

    // -- Log the hop-by-hop path --------------------------------
    if (logLevel >= LogLevel::INFO && fwdResult.path.size() > 1) {
        for (size_t i = 0; i < fwdResult.path.size() - 1; ++i) {
            logInfo("Router " + std::to_string(fwdResult.path[i])
                    + " forwarded packet to Router "
                    + std::to_string(fwdResult.path[i + 1]));
        }
    }

    // -- Build result -------------------------------------------
    SimulationResult result;
    result.source              = source;
    result.destination         = destination;
    result.path                = std::move(fwdResult.path);
    result.hopCount            = static_cast<int>(result.path.size()) - 1;
    result.latencyMicroseconds = elapsed.count();
    result.delivered           = fwdResult.delivered;
    result.failureReason       = std::move(fwdResult.failureReason);

    if (result.delivered) {
        logInfo("Packet delivered successfully");
    } else {
        logInfo("Packet DROPPED: " + result.failureReason);
    }

    return result;
}

// ====================================================================
// Batch simulation
// ====================================================================

BatchResult Simulator::runBatch(int numPackets, unsigned int seed) {
    const int n = network.getRouterCount();
    if (n < 2) {
        return {numPackets, 0, numPackets, 0, 0, 0, 0};
    }

    unsigned int effectiveSeed = seed;
    if (effectiveSeed == 0) {
        effectiveSeed = std::random_device{}();
    }
    std::mt19937 rng(effectiveSeed);
    std::uniform_int_distribution<int> nodeDist(0, n - 1);

    BatchResult batch{};
    batch.totalPackets = numPackets;
    batch.delivered    = 0;
```

```
    batch.dropped      = 0;
    batch.avgLatencyUs = 0.0;
    batch.minLatencyUs = std::numeric_limits<double>::max();
    batch.maxLatencyUs = 0.0;
    batch.avgHops      = 0.0;

    double totalLatency = 0.0;
    double totalHops    = 0.0;

    // Suppress per-hop logs during batch (use SILENT internally)
    LogLevel savedLevel = logLevel;
    logLevel = LogLevel::SILENT;

    for (int i = 0; i < numPackets; ++i) {
        int src = nodeDist(rng);
        int dst = nodeDist(rng);
        while (dst == src) {
            dst = nodeDist(rng);   // ensure src != dst
        }

        auto result = sendPacket(src, dst);

        if (result.delivered) {
            ++batch.delivered;
            totalLatency += result.latencyMicroseconds;
            totalHops    += result.hopCount;
            batch.minLatencyUs = std::min(batch.minLatencyUs,
                                          result.latencyMicroseconds);
            batch.maxLatencyUs = std::max(batch.maxLatencyUs,
                                          result.latencyMicroseconds);
        } else {
            ++batch.dropped;
        }
    }

    logLevel = savedLevel;  // restore original log level

    if (batch.delivered > 0) {
        batch.avgLatencyUs = totalLatency / batch.delivered;
        batch.avgHops      = totalHops    / batch.delivered;
    } else {
        batch.minLatencyUs = 0.0;
    }

    return batch;
}

// ============================================================================
// Pretty-printing
// ============================================================================

void Simulator::printResult(const SimulationResult& r) const {
    std::cout << "\n+-------------------------------------+\n";
    std::cout << "|        Simulation Result            |\n";
    std::cout << "+-------------------------------------+\n";

    std::cout << "|  Source:       Router " << r.source << "\n";
```

```
    std::cout << "|  Destination:  Router " << r.destination << "\n";
    std::cout << "|  Path:         ";
    for (size_t i = 0; i < r.path.size(); ++i) {
        if (i > 0) std::cout << " -> ";
        std::cout << r.path[i];
    }
    std::cout << "\n";

    std::cout << "|  Hop Count:    " << r.hopCount << "\n";
    std::cout << std::fixed << std::setprecision(3);
    std::cout << "|  Latency:      " << r.latencyMicroseconds << " us\n";
    std::cout << "|  Status:       "
              << (r.delivered ? "Delivered [OK]" : "DROPPED [X]") << "\n";
    if (!r.delivered) {
        std::cout << "|  Reason:       " << r.failureReason << "\n";
    }

    std::cout << "+-------------------------------------+\n";
}

void Simulator::printBatchResult(const BatchResult& b) const {
    std::cout << "\n+=====================================+\n";
    std::cout << "|      Batch Simulation Results       |\n";
    std::cout << "+=====================================+\n";

    std::cout << "|  Total Packets:   " << b.totalPackets << "\n";
    std::cout << "|  Delivered:       " << b.delivered << "\n";
    std::cout << "|  Dropped:         " << b.dropped << "\n";

    double successRate = (b.totalPackets > 0)
        ? (100.0 * b.delivered / b.totalPackets) : 0.0;
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "|  Success Rate:    " << successRate << "%\n";

    std::cout << std::fixed << std::setprecision(3);
    std::cout << "|  Avg Latency:     " << b.avgLatencyUs << " us\n";
    std::cout << "|  Min Latency:     " << b.minLatencyUs << " us\n";
    std::cout << "|  Max Latency:     " << b.maxLatencyUs << " us\n";
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "|  Avg Hops:        " << b.avgHops << "\n";

    std::cout << "+=====================================+\n";
}
```

Main Entry Point (src/main.cpp)

```
/**
 * @file main.cpp
 * @brief Network Routing Simulator ? demonstration entry point.
 *
 * Runs two demos:
 *   1. 20-router random connected graph  ? single packet + batch of 100
 *   2. 50-router random connected graph  ? single packet + batch of 500
 *
 * All routers exist as in-process objects.  Routing tables are generated
 * via BFS (pluggable strategy).  Timing measures raw C++ forwarding
```

```
 * overhead using std::chrono::steady_clock.
 */

#include <iostream>
#include "Simulator.h"

int main() {
    std::cout << "========================================\n";
    std::cout << "  Network Routing Simulator v1.0\n";
    std::cout << "  Baseline Public Routing Network\n";
    std::cout << "========================================\n\n";

    // ------------------------------------------------------
    // Demo 1: Small random topology (20 routers)
    // ------------------------------------------------------
    std::cout << "--- Demo 1: Random Topology (20 routers) ---\n\n";

    Simulator sim1(LogLevel::INFO);
    sim1.setupNetwork(20, TopologyType::RANDOM);

    // Single packet: Router 0 -> Router 19
    auto result1 = sim1.sendPacket(0, 19, "Hello Router 19!");
    sim1.printResult(result1);

    // Batch test: 100 random source/destination pairs
    std::cout << "\n--- Batch Simulation (100 packets, 20 routers) ---\n";
    auto batch1 = sim1.runBatch(100);
    sim1.printBatchResult(batch1);

    // ------------------------------------------------------
    // Demo 2: Medium random topology (50 routers)
    // ------------------------------------------------------
    std::cout << "\n--- Demo 2: Random Topology (50 routers) ---\n\n";

    Simulator sim2(LogLevel::SILENT);   // suppress per-hop logs at scale
    sim2.setupNetwork(50, TopologyType::RANDOM);

    // Single packet: Router 0 -> Router 49
    auto result2 = sim2.sendPacket(0, 49);
    sim2.printResult(result2);

    // Batch test: 500 random source/destination pairs
    std::cout << "\n--- Batch Simulation (500 packets, 50 routers) ---\n";
    auto batch2 = sim2.runBatch(500);
    sim2.printBatchResult(batch2);

    // ------------------------------------------------------
    // Demo 3: Ring topology (10 routers) ? deterministic path
    // ------------------------------------------------------
    std::cout << "\n--- Demo 3: Ring Topology (10 routers) ---\n\n";

    Simulator sim3(LogLevel::INFO);
    sim3.setupNetwork(10, TopologyType::RING);

    auto result3 = sim3.sendPacket(0, 5, "Ring test");
    sim3.printResult(result3);
```

```
    std::cout << "\n=======================================\n";
    std::cout << "  All demos completed successfully.\n";
    std::cout << "=======================================\n";

    return 0;
}
```

CMake Build Configuration (CMakeLists.txt)

```
cmake_minimum_required(VERSION 3.16)
project(RoutingSimulator VERSION 1.0 LANGUAGES CXX)

# -- C++17 Standard ------------------------------------------------
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)       # No GNU extensions ? ensures portability

# -- Executable ----------------------------------------------------
add_executable(routing_simulator
    src/main.cpp
    src/Packet.cpp
    src/Router.cpp
    src/RoutingTableGenerator.cpp
    src/Network.cpp
    src/Simulator.cpp
)

# -- Include Directories -------------------------------------------
target_include_directories(routing_simulator PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

# -- Compiler Warnings (optional but recommended) ---------------
if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
    target_compile_options(routing_simulator PRIVATE -Wall -Wextra -Wpedantic)
elseif(MSVC)
    target_compile_options(routing_simulator PRIVATE /W4)
endif()
```

Docker Build Configuration (Dockerfile)

```
# ============ Stage 1: Build ============
FROM debian:bookworm-slim AS builder

RUN apt-get update && apt-get install -y --no-install-recommends \
        build-essential \
        cmake \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy build config first (layer caching ? CMake config rarely changes)
COPY CMakeLists.txt .
COPY include/ include/
COPY src/ src/
```

```
RUN cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
    && cmake --build build --parallel

# ============ Stage 2: Runtime ============
FROM debian:bookworm-slim

RUN apt-get update && apt-get install -y --no-install-recommends \
        libstdc++6 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY --from=builder /app/build/routing_simulator .

CMD ["./routing_simulator"]
```